

Standalone Telemetry Worker Manual

2026-05-04

Standalone Telemetry Worker

A Cloudflare Worker dedicated to telemetry ingestion. Same payload schema and same backing storage (D1 + KV) as the Pages Function on the customer portal — different code surface.

URL

```
https://preston-check-telemetry.preston-check-edge.workers.dev/
```

This is the production endpoint that `lib/telemetry.sh` defaults to.

Why two endpoints

We have two implementations of the same telemetry logic:

1. **Standalone Worker** (this one) — `workers/telemetry/`, deployed via `wrangler deploy` to a `*.workers.dev` URL
2. **Pages Function** — `web/customer/functions/api/v1/telemetry/`, deployed alongside the customer portal at `app.preston-check.com/api/v1/telemetry`

Both write to the same D1 + KV instances. The standalone Worker is the canonical production endpoint; the Pages Function is an alternative for customers who prefer a `preston-check.com`-branded URL once the Pages Function regression is fully resolved.

Self-hosted Enterprise deployments typically prefer the standalone Worker pattern because it's a clean, single-purpose endpoint that doesn't share infrastructure with any UI.

Source

```
workers/telemetry/
  src/index.ts           TypeScript Worker
  wrangler.toml          project config + D1/KV bindings
  schema.sql             D1 schema
  README.md              deploy + dev notes
  .github/workflows/telemetry-deploy.yml  CI auto-deploy on push
```

How it works

The Worker exports a single `fetch` handler with three branches:

- `OPTIONS` — CORS preflight
- `POST` — schema-validate, rate-limit, write to D1+KV
- anything else → `405 method not allowed`

The full implementation is ~140 lines of TypeScript. Privacy-relevant properties (no source-code accepted, no IP stored, schema-strict validation) are auditable inline.

Bindings

The Worker requires three bindings configured in `wrangler.toml`:

```
[[d1_databases]]
binding      = "DB"
database_name = "preston-check-telemetry"
database_id   = "e206e1e4-1c78-4a5e-a983-bc47104d1b3c"

[[kv_namespaces]]
binding = "AGGREGATE"
id      = "330983ec5b464dab8ae2f338d40512f5"

[vars]
RATE_LIMIT_PER_HOUR = "1000"
ALLOW_ORIGIN         = "*"

```

(IDs are public identifiers, not credentials. The token to access them lives only in repo secrets.)

Deploying

CI auto-deploys on every push that touches `workers/telemetry/**`. Manual deploy:

```
gh workflow run telemetry-deploy.yml --repo preston-check/preston-check
```

Or from a local checkout (requires `CLOUDFLARE_API_TOKEN` + `CLOUDFLARE_ACCOUNT_ID` env vars):

```
cd workers/telemetry
wrangler deploy
```

Provisioning from scratch

For a fresh self-hosted setup:

```
# 1. Create the D1 database
wrangler d1 create preston-check-telemetry
# (capture the returned database_id and put it in wrangler.toml)

# 2. Create the KV namespace
wrangler kv namespace create AGGREGATE
# (capture the namespace id and put it in wrangler.toml)

# 3. Apply the schema (remote - runs on the actual Cloudflare D1)
wrangler d1 execute preston-check-telemetry --remote \
  --file=workers/telemetry/schema.sql

# 4. Deploy the Worker
wrangler deploy
```

The Worker is reachable at `<worker-name>.<account-subdomain>.workers.dev` within seconds. Optional: bind a custom domain in the Cloudflare dashboard if your account's zone is on Cloudflare.

Health check

```
HASH=$(printf 'test' | shasum -a 256 | cut -d' ' -f1)
curl -s -X POST 'https://preston-check-telemetry.preston-check-edge.workers.dev/' \
  -H "Content-Type: application/json" \
  -d '{"version":"1.7.5","tier":"free","lang":"bash","repo_hash":"$HASH","pass":1,"fail":0,"warn":0,"skip":0,05-04T12:00:00Z}'
# Expected: {"ok":true}
```

Cross-links

- **Telemetry endpoint manual:** `docs/manuals/telemetry-endpoint.md`

- **D1 dataset manual:** [docs/manuals/d1-dataset.md](#)
- **Privacy + telemetry overview:** [docs/telemetry.md](#)